

Haskell Types Practice: October 8, 2026

This worksheet follows Questions 2 and 3 of the Spring 2024 CMSC 433 midterm, with different expressions and target types. The two main sections are worth 20 points each. Optional challenges follow.

Editing: Replace each `TODO` in this source file with your answer. Answer boxes accept ordinary Haskell syntax.

Question 2: What is the type? (20 points)

For each expression, give its **most general Haskell type**, including all necessary typeclass constraints, or write `ill-typed` and briefly explain the mismatch. **Exam convention: treat all raw numeric literals as `Int`.** Give the most general type under that convention. Use the reference appendix for library signatures.

(a) `\x y -> (y, x, x)`

`a -> b -> (b, a, a)`

(b) `\x y -> if x < y then show x else show (y, x)`

`(Ord a, Show a) => a -> a -> String`

(c) `\x xs -> x : reverse xs ++ [x]`

`a -> [a] -> [a]`

(d) `foldr (const False)`

`ill typed`

(e) `(, 'q')`

`a -> (a, Char)`

(f) `(,) ['q']`

`b -> ([Char], b)`

(g) `reverse . map Just`

`[a] -> [Maybe a]`

(h) `\xs -> filter (>= 7) (reverse xs)`

`[Int] -> [Int]`

(i) `let keep x = x in (keep 'q', keep [False])`

`(Char, [Bool])`

(j) `fmap (fmap (== 'q')) [Nothing, Just 'r']`

`[Maybe Bool]`

Question 3: Give an expression matching the type (20 points)

For each type below, write a Haskell expression that has that type. Do not write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You may use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Bool -> Maybe Bool`

```
\x -> if x then Just x else Nothing
```

(b) `a -> [b]`

```
\x -> []
```

(c) `(Char -> Int -> Bool) -> [Char] -> [Int] -> [Bool]`

```
liftA2
```

(d) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe (c, b)`

```
\f -> liftA2 (\x y -> (f x y, y))
```

(e) `(a -> b) -> (a -> Bool) -> [a] -> [b]`

```
\f f2 xs -> map f (filter f2 xs)
```

(f) `Maybe a -> (a -> [b]) -> [(b, a)]`

```
\ma f -> maybe [] (\x -> map (\y -> (y, x)) (f x)) ma
```

(g) `Eq a => [a] -> a -> [Bool]`

```
\(x:xs) y -> if y == x then [True] else [False]
```

(h) `Show a => Maybe a -> IO String`

```
\ma -> return (show ma)
```

(i) `(a -> b -> c) -> (b, a) -> c`

```
\f (b, a) -> f a b
```

(j) `((a, b) -> c) -> b -> c`

```
undefined
```

Optional challenges

These are additional practice and are not part of the 40-point worksheet.

(a) Give the most general type or explain the type error. Compare the reason for your answer with Question 2(i).

```
\keep -> (keep 'q', keep [False])
```

```
TODO
```

- (b) Give the most general type or explain the type error. Identify the structures acted on by both occurrences of `fmap`.

```
fmap (fmap not) [Just True, Nothing, Just 'q']
```

TODO

- (c) Give the most general type. Which type variable belongs to the input container, and which belongs to the accumulator?

```
foldl (flip (:)) []
```

TODO

- (d) Give an expression of this type. Apply the function to each input, discard absent results, and pair each present result with its original input. Preserve input order, including duplicates.

```
(a -> Maybe b) -> [a] -> [(a, b)]
```

TODO

Reference appendix

These signatures use the generalized `Foldable` versions from the old exam's appendix. `String` is a synonym for `[Char]`. Tuple sections, such as `(, False)`, are allowed as in the exam; enable `TupleSections` if checking them in `GHCi`.

```
data Bool = False | True
data Maybe a = Nothing | Just a
```

```
id :: a -> a
const :: a -> b -> a
flip :: (a -> b -> c) -> b -> a -> c
(.) :: (b -> c) -> (a -> b) -> a -> c
(,) :: a -> b -> (a, b)
```

```
not :: Bool -> Bool
(==), (/=) :: Eq a => a -> a -> Bool
(<), (<=), (>), (>=) :: Ord a => a -> a -> Bool
(+), (-), (*) :: Num a => a -> a -> a
show :: Show a => a -> String
```

```
(:) :: a -> [a] -> [a]
(++) :: [a] -> [a] -> [a]
reverse :: [a] -> [a]
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
```

```
fmap :: Functor f => (a -> b) -> f a -> f b
pure :: Applicative f => a -> f a
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
return :: Monad m => a -> m a
(>=>) :: Monad m => m a -> (a -> m b) -> m b
(>>) :: Monad m => m a -> m b -> m b
```

```
maybe :: b -> (a -> b) -> Maybe a -> b
maybeToList :: Maybe a -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

```
print :: Show a => a -> IO ()
putStrLn :: String -> IO ()
```

Numeric literals: For this worksheet, raw numbers have type `Int`, following the TA's exam clarification. When a type is fixed to `Int`, omit constraints such as `Ord Int` and `Num Int`, since those instances are already known. Library signatures above remain general. In GHCi, annotate a literal with `:: Int` to reproduce the exam convention when checking a type.

Constraints: `Ord a` implies `Eq a`; it does not imply `Num a` or `Show a`. Lists and `Maybe` are instances of `Functor`, `Applicative`, and `Monad`. They are also `Foldable`. Tuples and `Maybe a` have `Show` instances when their contained types have `Show` instances.